

Laboratorio di Programmazione

Lezione 5: Funzioni

1 Qual è l'output?	2
2 Qual è l'output?	3
3 Qual è l'output?	4
4 Trova l'errore	5
5 Trova l'errore	6
6 Cerchio	7
7 Tabelline	8
8 Tabelline 2	9
9 Tabelline 3	10
10 Radice quadrata	11
11 Scacchiera	12
12 Numeri primi	13
13 Numeri primi gemelli	14
14 Numeri perfetti	15
15 Numeri abbondanti	16
16 Numeri amichevoli	17
17 Terna pitagorica	18
18 Scala	19
19 Scala 2	20

1 Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func procedura1() {
6     fmt.Println("Hello world!")
7 }
8
9 func main() {
10     procedura1()
11 }
```

2 Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func funzione(x int) int {
6     return x + 5
7 }
8
9 func main() {
10     var x int = 10
11     fmt.Println(funzione(x))
12 }
```

3 Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func funzione1(x int) int {
6     return x * 10
7 }
8
9 func funzione2(x int) (y int) {
10     y = x * 10
11     return
12 }
13
14 func main() {
15     fmt.Println(funzione1(5), funzione2(3))
16 }
```

4 Trova l'errore

Questo programma dovrebbe stampare 20 40, ma non lo fa. Correggetelo.

```
1 package main
2
3 import "fmt"
4
5 func test(x, y int) (int, int) {
6     return 2 * x, 2 * y
7 }
8
9 func main() {
10     var x, y int = 10, 20
11     test(x, y)
12     fmt.Println(x, y)
13 }
```

5 Trova l'errore

Questo programma contiene degli errori. Correggetelo.

```
1 package main
2
3 import "fmt"
4
5 func test(x int) y int, z int
6 {
7     var y int = x + 1
8     var z int = x + 2
9     return
10 }
11
12 func main() {
13     var a, b int
14     a, b = test(10)
15     fmt.Println(a, b)
16 }
```

6 Cerchio

Scrivere un programma che legga da standard input un numero reale r e stampi i valori di circonferenza e area di un cerchio di raggio r . Oltre alla funzione `main()`, il programma deve definire e utilizzare le seguenti funzioni:

- `CalcolaArea(r float64) float64`.
- `CalcolaCirconferenza(r float64) float64`.

Esempio d'esecuzione:

```
$ go run cerchio.go
Inserisci il raggio del cerchio: 10.5
Area del cerchio: 346.36059005827474
Circonferenza del cerchio: 65.97344572538566
```

```
$ go run cerchio.go
Inserisci il raggio del cerchio: 3.6
Area del cerchio: 40.71504079052372
Circonferenza del cerchio: 22.61946710584651
```

7 Tabelline

Scrivere un programma che legge da standard input un numero intero e stampa la tabellina corrispondente solo se il numero inserito è in $\{1, \dots, 9\}$. Oltre alla funzione `main()`, il programma deve definire e utilizzare le seguenti funzioni:

- `Tabellina(n int)` che stampa la tabellina di n , ma solo se $1 \leq n \leq 9$.

Esempio d'esecuzione:

```
$ go run tabellina.go
Inserisci un numero: 4
Tabellina del 4: 0 4 8 12 16 20 24 28 32 36 40
```

```
$ go run tabellina.go
Inserisci un numero: 6
Tabellina del 6: 0 6 12 18 24 30 36 42 48 54 60
```

```
$ go run tabellina.go
Inserisci un numero: 13
```

```
$ go run tabellina.go
Inserisci un numero: -1
```


8 Tabelline 2

Adattate la soluzione dell'esercizio 7 in modo da stampare `Numero non valido` se il numero inserito non è in $\{1, \dots, 9\}$. A tal scopo modificate la funzione `Tabellina` in modo che restituisca un valore booleano che è `false` se e solo se `n` non è in $\{1, \dots, 9\}$.

Esempio d'esecuzione:

```
$ go run tabellina.go
Inserisci un numero: 4
Tabellina del 4: 0 4 8 12 16 20 24 28 32 36 40
```

```
$ go run tabellina.go
Inserisci un numero: 6
Tabellina del 6: 0 6 12 18 24 30 36 42 48 54 60
```

```
$ go run tabellina.go
Inserisci un numero: 13
Numero non valido.
```

```
$ go run tabellina.go
Inserisci un numero: -1
Numero non valido.
```

9 Tabelline 3

Adattate la soluzione dell'esercizio 7 così che il programma chieda ripetutamente l'intero n ; se è in $\{1, \dots, 9\}$ ne stampa la tabellina, altrimenti stampa Programma terminato ed esce.

Esempio d'esecuzione:

```
$ go run tabellina.go
Inserisci un numero: 6
Tabellina del 6: 0 6 12 18 24 30 36 42 48 54 60
Inserisci un numero: 4
Tabellina del 4: 0 4 8 12 16 20 24 28 32 36 40
Inserisci un numero: 13
Programma terminato.
```

10 Radice quadrata

Scrivere un programma che legge da standard input un numero reale x e stampa il valore della radice quadrata di x solo se $x \geq 0$. Se invece $x < 0$ il programma deve stampare:

Il valore inserito non appartiene al dominio della funzione

Oltre alla funzione `main()`, il programma deve definire e utilizzare una funzione

```
RadiceQuadrata(x float64) (float64, bool)
```

che restituisce la radice di x e `true` se $x \geq 0$, e restituisce `0` e `false` se $x < 0$. Potete usare la funzione `Sqrt` del package `math`.

Esempio d'esecuzione:

```
$ go run radice.go
Inserisci un numero: 10
Radice quadrata: 3.1622776601683795
```

```
$ go run radice.go
Inserisci un numero: 4
Radice quadrata: 2
```

```
$ go run radice.go
Inserisci un numero: -1
Il valore inserito non appartiene al dominio della funzione
```

11 Scacchiera

Scrivere un programma che legge da standard input un intero n e, come mostrato sotto, stampa a video una schacchiera di n righe ed n colonne utilizzando i caratteri $*$ (asterisco) e $+$ (più). Oltre a `main()` il programma deve definire e utilizzare le seguenti funzioni:

- `StampaRigaInizioAsterisco(n int)` che stampa una riga lunga n che inizia con $*$;
- `StampaRigaInizioPiù(n int)` che stampa una riga lunga n che inizia con $+$;
- `StampaScacchiera(n int)` che stampa l'intera scacchiera, usando le funzioni sopra (se $n \leq 0$ non stampa alcunché).

Esempio d'esecuzione:

```
$ go run scacchiera.go
Inserisci la dimensione: 4
*+*+
+*+*
*+*+
+*+*
```

```
$ go run scacchiera.go
Inserisci la dimensione: 6
*+*+*+
+*+*+*
*+*+*+
+*+*+*
*+*+*+
+*+*+*
```

```
$ go run scacchiera.go
Inserisci la dimensione: -1
```

```
$ go run scacchiera.go
Inserisci la dimensione: 0
```

12 Numeri primi

Un intero n è *primo* se $n \geq 2$ e se è divisibile solo per se stesso e per 1. Scrivere un programma che legge da standard input un intero N e stampi, in ordine crescente, tutti i numeri primi n strettamente minori di N . Se $N \leq 0$ il programma deve stampare:

La soglia inserita non è positiva

Oltre a `main()`, il programma deve definire e utilizzare le seguenti funzioni:

- `ÈPrimo(n int) bool` che decide se n è primo.
- `NumeriPrimi(N int)` che stampa tutti i primi in $\{2, \dots, N\}$, usando `ÈPrimo()`.

Esempio d'esecuzione:

```
$ go run numeri_primi.go
Inserisci un numero: -3
La soglia inserita non è positiva
```

```
$ go run numeri_primi.go
Inserisci un numero: 5
Numeri primi inferiori a 5
2 3
```

```
$ go run numeri_primi.go
Inserisci un numero: 12
Numeri primi inferiori a 12
2 3 5 7 11
```

13 Numeri primi gemelli

Due primi p, q sono *gemelli* se $p = q + 2$. Scrivere un programma che legge da standard input un intero N e stampa, in ordine crescente, tutte le coppie di primi gemelli con $p < N$. Se $N < 0$ il programma deve stampare “La soglia inserita non è positiva”. Oltre a `main()`, il programma deve definire e utilizzare le seguenti funzioni:

- `ÈPrimo(n int) bool` come nell’esercizio 12
- `PrimiGemelli(N int)` che, usando `ÈPrimo()`, stampa le coppie di primi gemelli come richiesto sopra.

Esempio d’esecuzione:

```
$ go run numeri_primi_gemelli.go
Inserisci un numero: -4
La soglia inserita non è positiva
```

```
$ go run numeri_primi_gemelli.go
Inserisci un numero: 10
Numeri primi gemelli inferiori a 10
(3,5) (5,7)
```

```
$ go run numeri_primi_gemelli.go
Inserisci un numero: 20
Numeri primi gemelli inferiori a 20
(3,5) (5,7) (11,13) (17,19)
```

14 Numeri perfetti

Un numero naturale n è *perfetto* se n è la somma dei suoi divisori propri (un divisore d di n è *proprio* se $d < n$). Ad esempio, 6 è perfetto perché i suoi divisori propri sono 1, 2, 3, e la loro somma è 6. Scrivere un programma che legge da standard input un intero n e decida se è perfetto. Oltre a funzione `main()`, il programma deve definire e usare le seguenti funzioni:

- `SommaDivisoriPropri(n int) int` che calcola la somma dei divisori propri di n . Tale somma vale 0 se n non ha divisori propri.
- `ÈPerfetto(n int) bool` che decide se n è perfetto, usando `SommaDivisoriPropri()`.

Esempio d'esecuzione:

```
$ go run numero_perfetto.go
Inserisci un numero: 0
0 non è perfetto
```

```
$ go run numero_perfetto.go
Inserisci un numero: 1
1 non è perfetto
```

```
$ go run numero_perfetto.go
Inserisci un numero: 6
6 è perfetto
```

```
$ go run numero_perfetto.go
Inserisci un numero: 28
28 è perfetto
```

15 Numeri abbondanti

Un numero naturale n è *abbondante* se è minore della somma dei suoi divisori propri. Per esempio, 12 è abbondante perché la somma dei suoi divisori propri è $1+2+3+4+6 = 16 > 12$. Adattate il programma dell'esercizio 14 in modo da prendere da standard input un intero N e stampare tutti i numeri abbondanti minori di N . Oltre a `main()`, il programma deve definire e usare le seguenti funzioni:

- `SommaDivisoriPropri(n int) int` come nell'esercizio 14.
- `ÈAbbondante(n int) bool` che decide se n è abbondante.
- `NumeriAbbondanti(N int)` che stampa, in ordine crescente, tutti i numeri abbondanti minori o uguali a N .

Esempio d'esecuzione:

```
$ go run numeri_abbondanti.go
Inserisci un numero: 20
Numeri abbondanti: 12 18
```

```
$ go run numeri_abbondanti.go
Inserisci un numero: 30
Numeri abbondanti: 12 18 20 24
```

```
$ go run numeri_abbondanti.go
Inserisci un numero: -3
La soglia inserita non è positiva
```


16 Numeri amichevoli

Una coppia di numeri naturali (n, m) con $n \leq m$ è *amichevole* se la somma dei divisori propri di ciascuno è uguale all'altro. Per esempio $(220, 284)$ è amichevole, perché

$$284 = 1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110$$

$$220 = 1 + 2 + 4 + 71 + 142$$

Scrivere un programma che legge da standard input un intero N e stampa tutte le coppie amichevoli (n, m) tali che $n, m \leq N$. Se $N \leq 0$ il programma deve stampare “La soglia inserita non è p
Oltre a `main()`, il programma deve definire e usare le seguenti funzioni:

- `SommaDivisoriPropri(n int) int` come nell'esercizio 14.
- `SonoAmichevoli(n, m int) bool` che decide se n, m sono amichevoli.
- `NumeriAmichevoli(N int)` che stampa tutte le coppie di amichevoli come descritto sopra, per valore crescente di n e, in subordine, per valore crescente di m .

Esempio d'esecuzione:

```
$ go run numeri_amichevoli.go
Inserisci un numero: 300
Numeri amichevoli inferiori a 300
(220,284)
```

```
$ go run numeri_amichevoli.go
Inserisci un numero: 0
La soglia inserita non è positiva
```

17 Terna pitagorica

Una tripla (a, b, c) di naturali è una *terna pitagorica* se $a^2 + b^2 = c^2$. Scrivete un programma che legge da standard input un intero N e stampa tutte le terne pitagoriche (a, b, c) con $a, b, c \leq N$. Oltre a `main()` il programma deve definire e usare le seguenti funzioni:

- `Pitagorica(a int, b int, c int) bool` che decide se (a, b, c) è una terna pitagorica.
- `TernePitagoriche(N int)` che stampa tutte le terne pitagoriche come chiesto sopra.

Esempio d'esecuzione:

```
$ go run terna_pitagorica.go
Inserisci la soglia: 10
Terne pitagoriche:
(3, 4, 5)
(4, 3, 5)
```

```
$ go run terna_pitagorica.go
Inserisci la soglia: 20
Terne pitagoriche:
(3, 4, 5)
(4, 3, 5)
(5, 12, 13)
(6, 8, 10)
(8, 6, 10)
(8, 15, 17)
(9, 12, 15)
(12, 5, 13)
(12, 9, 15)
(15, 8, 17)
```

Bonus: raffinate la soluzione evitando di stampare doppioni, cioè evitando di stampare sia (a, b, c) che (b, a, c) .

18 Scala

Scrivete un programma che legge da standard input un intero n e, come mostrato sotto, stampa a video una scala di asterischi, secondo queste regole:

- La scala è formata da n gradini.
- Ciascun gradino è largo 3 simboli e alto 2 simboli.
- Ogni gradino è rientrato a destra di 2 spazi rispetto al precedente (notate che il primo gradino non ha alcuna rientranza).

Se $n \leq 0$ il programma deve stampare “Dimensione non sufficiente” e uscire. Oltre a `main()`, devono essere definite e usate le seguenti funzioni:

- `StampaGradino(n int)` che stampa il gradino n (il gradino 0 è il primo dall’alto).
- `StampaScala(N int)` che stampa la scala di N gradini.

Esempio d’esecuzione:

```
$ go run scala.go
Inserisci il numero dei gradini: 3
***
 *
***
 *
***
 *
```

```
$ go run scala.go
Inserisci il numero dei gradini: 0
Dimensione non sufficiente
```

```
$ go run scala.go
Inserisci il numero dei gradini: 2
***
 *
***
 *
```

19 Scala 2

Adattate la soluzione dell'esercizio 18 in modo da stampare una scala ascendente invece che discendente, come mostrato sotto.

Esempio d'esecuzione:

```
$ go run scala.go
Inserisci il numero dei gradini: 3
  ***
 *
***
 *
***
*
```

```
$ go run scala.go
Inserisci il numero dei gradini: 0
Dimensione non sufficiente
```

```
$ go run scala.go
Inserisci il numero dei gradini: 2
  ***
 *
***
*
```